

Agentic Generation Script Engine (v7)

Bidirectional Orchestration Suite • Retroactive Continuity & Rollback Engine

V7 Architectural Enhancements:

- **Retroactive Back-Propagation Layer:** Allows human artistic changes to be targeted backwards in time to previous scene indexes in the dynamic JSON structure.
- **State Cache Decoupling:** Programmatically updates and clears targeted historical records in `pipeline_state.json` for seamless context timeline re-rendering.
- **Global-Forward Synchronization:** Automatically updates root configuration keys while executing code rewind loops to keep upstream updates matched with future outputs.

Master Prompt Payload

"Act as an elite Systems Architect and Python Engineer specializing in stateful media pipelines. Write a complete, robust, production-ready Python automation script that reads a structured movie production JSON schema and executes an iterative, self-correcting video generation pipeline targeting Google Veo 3.1 and a custom Suno API proxy, featuring bidirectional state rollback.

The script must implement a multi-stage critique and refine loop utilizing the ``google-genai`` Python SDK for Veo 3.1, a Gemini API model for automated evaluation, and an interactive CLI state that handles three-tiered forward feedback and retroactive back-propagation.

Pipeline Infrastructure & State Persistence

1. Maintain progress tracking via a local `'pipeline_state.json'` file.
2. Hold the master movie blueprint JSON object fully in memory as a live, bidirectional state machine.

Pipeline Stage 1, 2, & 3: Generation, Evaluation, and Muxing

(Execute parallel Suno proxy requests, process concurrent Veo 3.1 video generation loops with context ID extension tracking, run automated Gemini vision QA passes, and execute localized FFmpeg scene mixing).

Pipeline Stage 4: Interactive Bidirectional Critic Gate

Right after Stage 3 assembles a scene file, pause execution and present this interactive menu:

1. Prompt the user: 'Review Scene [X]. Press [A] to Approve, [F] for Forward Feedback, [R] for Retroactive Rollback, or [Q] to Save & Quit.'
2. ****If the user inputs [F] (Forward):**** Process Local Clip, Cascading Scene, or Global Forward Scoping variables as defined in V6 rules.
3. ****If the user inputs [R] (Retroactive Rollback):****
 - Capture the written feedback string (e.g., 'Character_A must wear a black leather jacket').
 - Prompt the user for the target anchor: 'Enter the past scene number where this modification should begin:' (e.g., Target Scene Y).
 - ****Retroactive Propagation Engine:****
 - ****JSON Modification Loop:**** Programmatically loop backward through the live in-memory JSON structure from the current scene down to Target Scene Y. Append the feedback modifier string directly into the `'visual_prompt'` fields of all clips within

those scene blocks.

- ****Global Variables Sync:**** Update the root ``global_production_variables.character_seed_tokens`` to include the modifier so all future scenes inherit it too.
- ****State State Rollback:**** Open 'pipeline_state.json' and programmatically purge the completion records, asset links, and status flags for all scenes from Target Scene Y up to the current scene index.
- ****Pipeline Index Jump:**** Explicitly reset the global execution index pointer back to Target Scene Y. Break the current execution block and loop back to Stage 1 to begin re-rendering the historical timeline automatically.

Pipeline Stage 5: Global Film Mastering

(Stitch all approved scene files sequentially via FFmpeg using a 1.5-second 'acrossfade' parameter block to generate the final movie).

Ensure the code structure features robust file IO exception handling when modifying the state cache, and clear console logs tracking the state rollback boundaries."